

Computer Networks

Here is an example of code that can be used in the context of computer networks:

```
import socket

def get_local_ip():
    """
    Function to get the local IP address of the machine
    """
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        # doesn't have to be reachable
        s.connect(('10.255.255.255', 1))
        IP = s.getsockname()[0]
    except:
        IP = '127.0.0.1'
    finally:
        s.close()
    return IP

print(get_local_ip())
```

Unit 1 - Introductory Concepts of Computer Networks and Physical Layer

A computer network is a group of interconnected devices that can communicate with each other
The physical layer is the first layer of the OSI model and is responsible for the transmission of data

Example of a simple computer network:

```
devices = ['computer1', 'computer2', 'printer', 'router']

connections = {
    'computer1': ['router'],
    'computer2': ['router'],
    'printer': ['router'],
    'router': ['computer1', 'computer2', 'printer']
}

def send_data(sender, receiver, data):
    if receiver in connections[sender]:
        print(f"{sender} sent data to {receiver}: {data}")
    else:
        print(f"{sender} cannot send data to {receiver}")

send_data('computer1', 'printer', 'print this document')
```

Introductory Concepts of Computer Networks

A computer network is a group of interconnected devices that can communicate with each other and share resources. The devices can be computers, printers, servers, routers, switches, and other networking equipment. The communication between devices is facilitated by various networking protocols, which define the rules and standards for data transmission.

There are several types of computer networks, including Local Area Networks (LANs), Wide Area Networks (WANs), and the Internet. LANs are typically used to connect devices within a small geographic area, such as a home or office. WANs, on the other hand, are used to connect LANs over larger geographic distances. The Internet is the largest WAN, connecting millions of computers and networks worldwide.

Computer networks can be wired or wireless. Wired networks use cables to connect devices, while wireless networks use radio waves or infrared signals. Wireless networks are becoming increasingly popular due to their convenience and flexibility.

In summary, computer networks allow devices to communicate and share resources, and they come in various types and configurations. Understanding the basics of computer networks is essential for anyone working in the field of information technology.

Goals and applications of networks and protocols

The primary goal of networks and protocols is to enable communication between different devices and systems. This communication can take many forms, including the transfer of data, voice, and video. Networks and protocols facilitate this communication by providing a standardized way for devices to exchange information.

Some common applications of networks and protocols include:

- **File sharing:** Networks allow multiple users to access and share files stored on a central server.
- **Email:** Email protocols enable users to send and receive messages over a network.
- **Web browsing:** The HTTP protocol allows users to access and view web pages over the internet.
- **Streaming media:** Protocols such as RTP and RTSP enable the streaming of audio and video content over a network.
- **Online gaming:** Many online games use specialized protocols to enable multiplayer gameplay over a network.

These are just a few examples of the many applications of networks and protocols. By providing a standardized way for devices to communicate, networks

and protocols enable a wide range of services and applications that we use every day.

Categories of networks in computer networks

There are several categories of networks in computer networks, including:

1. **Local Area Network (LAN):** A LAN is a network that connects computers and other devices in a relatively small area, such as a home, school, or office building.
2. **Wide Area Network (WAN):** A WAN is a network that covers a large geographic area, such as a city, country, or even the entire world.
3. **Metropolitan Area Network (MAN):** A MAN is a network that connects computers and other devices in a metropolitan area, such as a city or a large campus.
4. **Personal Area Network (PAN):** A PAN is a network that connects devices in close proximity, such as a person's phone, laptop, and smart-watch.
5. **Storage Area Network (SAN):** A SAN is a network that provides access to consolidated, block-level data storage.
6. **Virtual Private Network (VPN):** A VPN is a network that uses the internet to provide remote access to a centralized organizational network.

Organization of the Internet

- The Internet is a loosely-organized international collaboration of autonomous, interconnected networks that supports host-to-host communication through voluntary adherence to open protocols and procedures defined by Internet Standards.
- There are also many isolated internets, i.e., sets of interconnected networks, which are not connected to the Internet but use the Internet Standards.
- A final level of organization of the Internet is at the level of individual sites and communities which develop their own rules and terms of usage.
- Many online communities, in fact, take on structures that are familiar in other organizations.
- The Internet, and the networks and technologies through which it operates, are controlled and organized at four particular levels: computer and network hardware, Internet access and Internet service providers (ISPs), navigation within the Internet.
- Internet Society is a non-profit organization empowering people to keep the Internet a force for good: open, globally connected, secure, and trustworthy.

- Through members, chapters, special interest groups, and partners, they are the hub of the largest global network of people and organizations working to build the Internet for everyone.

ISP

ISP stands for Internet Service Provider. An ISP is a company that provides internet access to customers. ISPs can provide internet access through various technologies such as DSL, cable modem, fiber, wireless, satellite, and dial-up.

- ISPs connect customers to the internet by providing them with a connection to their network.
- ISPs can offer different types of internet connections, such as broadband or dial-up.
- ISPs can also offer additional services such as email, web hosting, and virtual private networks (VPNs).
- ISPs are regulated by government agencies to ensure that they provide reliable and affordable internet access to customers.
- Customers can choose from different ISPs based on factors such as price, speed, and availability.

Network structure with reference to Computer Networks

A computer network is a group of interconnected devices that can communicate with each other and share resources. The structure of a computer network refers to the way these devices are connected and how they interact with each other.

There are several common network structures, including:

- **Star topology:** In a star network, all devices are connected to a central hub or switch. Data is transmitted from one device to the hub, which then forwards it to the destination device.
- **Bus topology:** In a bus network, all devices are connected to a common cable, called the bus. Data is transmitted along the bus and all devices can receive it, but only the intended recipient processes it.
- **Ring topology:** In a ring network, all devices are connected in a closed loop. Data is transmitted from one device to the next in the loop until it reaches its destination.
- **Mesh topology:** In a mesh network, each device is connected to multiple other devices. This provides multiple paths for data to travel, increasing the network's resilience to failures.

Each network structure has its own advantages and disadvantages, and the choice of structure depends on factors such as the size of the network, the type of data being transmitted, and the desired level of reliability and performance.

Network architecture with reference to Computer Networks

Network architecture refers to the design and structure of a computer network. It includes the hardware and software components, the protocols and standards used for communication, and the overall organization of the network. The architecture of a network can be divided into different layers, with each layer responsible for a specific function. The most common network architecture model is the OSI (Open Systems Interconnection) model, which consists of seven layers: Physical, Data Link, Network, Transport, Session, Presentation, and Application. Each layer has its own set of protocols and standards that enable communication between devices on the network. The design of a network architecture depends on various factors such as the size and complexity of the network, the type of data being transmitted, and the security requirements. A well-designed network architecture can improve the performance, reliability, and scalability of the network.

Layering Principles with Reference to Network Architecture in Computer Networks

Layering is a design principle that is used to structure computer networks into multiple layers, each of which provides a specific set of services to the layer above it. This approach simplifies the design and implementation of complex network systems by breaking them down into smaller, more manageable components.

In the context of network architecture, layering is typically implemented using a protocol stack, where each layer in the stack corresponds to a specific level of abstraction. For example, the OSI (Open Systems Interconnection) model is a widely used reference model for network architecture that defines seven layers, each with its own set of functions and protocols.

The layers in the OSI model, from lowest to highest, are:

1. Physical Layer: This layer is responsible for the transmission of raw data bits over a physical medium, such as a cable or wireless link.
2. Data Link Layer: This layer provides reliable transmission of data frames between two nodes connected by a physical link.
3. Network Layer: This layer provides routing and forwarding of data packets between nodes in a network.
4. Transport Layer: This layer provides end-to-end communication services, such as error recovery and flow control, between two nodes in a network.
5. Session Layer: This layer manages the establishment, maintenance, and termination of sessions between two nodes in a network.
6. Presentation Layer: This layer provides data representation and encoding services, such as data compression and encryption, to ensure that data can be exchanged between nodes in a network.
7. Application Layer: This layer provides high-level services, such as file transfer and email, to applications running on nodes in a network.

By using a layered approach, network designers can focus on the specific requirements of each layer, without having to worry about the details of the other layers. This makes it easier to develop, test, and maintain complex network systems. Additionally, layering allows for the reuse of common protocols and services across different network architectures, which can reduce development costs and improve interoperability between different systems.

Services in Networks Architecture in Computer Networks

In computer networks, the architecture is divided into layers, with each layer providing a specific set of services to the layer above it. The services provided by each layer can be categorized into two types: connection-oriented and connectionless.

Connection-oriented services establish a dedicated connection between two devices before any data is transmitted. This type of service is reliable and ensures that data is delivered in the correct order. An example of a connection-oriented service is the Transmission Control Protocol (TCP).

Connectionless services, on the other hand, do not establish a dedicated connection between devices. Instead, data is transmitted as individual packets, with each packet being routed independently. This type of service is less reliable, as packets may be lost or delivered out of order. An example of a connectionless service is the User Datagram Protocol (UDP).

In addition to these two types of services, there are also other services provided by the network architecture, such as error control, flow control, and congestion control. These services help to ensure that data is transmitted reliably and efficiently across the network.

Protocols and Standards in Networks Architecture in Computer Networks

In computer networks, protocols and standards are essential for ensuring that devices can communicate with each other and exchange data. Protocols are sets of rules and procedures that define how data is transmitted and received over a network. Standards, on the other hand, are agreed-upon specifications that ensure that different devices and systems can work together.

Some common protocols used in computer networks include the Transmission Control Protocol (TCP), the User Datagram Protocol (UDP), and the Internet Protocol (IP). These protocols define how data is transmitted over the internet and are essential for ensuring that data can be sent and received reliably.

Standards, such as the Ethernet standard and the Wi-Fi standard, define the physical and technical specifications for devices and systems used in computer networks. These standards ensure that different devices can communicate with each other and exchange data, even if they are made by different manufacturers.

In summary, protocols and standards are essential for ensuring that computer networks can function effectively and that data can be transmitted and received reliably. They provide a common framework for communication and ensure that different devices and systems can work together.

The OSI reference model in Computer Networks

The Open Systems Interconnection (OSI) model is a conceptual framework used to describe the functions of a networking system. The OSI model characterizes computing functions into a universal set of rules and requirements in order to support interoperability between different products and software.

The model partitions a communication system into seven abstraction layers. The original version of the model defined the following seven layers:

1. Physical Layer
2. Data Link Layer
3. Network Layer
4. Transport Layer
5. Session Layer
6. Presentation Layer
7. Application Layer

Each layer serves the layer above it and is served by the layer below it. For example, a layer that provides error-free communications across a network provides the path needed by applications above it, while it calls the next lower layer to send and receive packets that comprise the contents of that path. Two instances at the same layer are visualized as connected by a horizontal connection in that layer.

The model is a product of the Open Systems Interconnection project at the International Organization for Standardization (ISO).

TCP/IP protocol suite in Computer Networks

The TCP/IP protocol suite is a set of communication protocols used for the Internet and other similar networks. It is named after the two most important protocols in the suite: the Transmission Control Protocol (TCP) and the Internet Protocol (IP).

Here is an example of how the TCP/IP protocol suite can be implemented in code:

```
import socket

def send_data(data, destination_ip, destination_port):
    # create a socket object
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```

    # connect to the destination
    sock.connect((destination_ip, destination_port))

    # send the data
    sock.sendall(data)

    # close the socket
    sock.close()

def receive_data(port):
    # create a socket object
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # bind the socket to the port
    server_address = ('', port)
    sock.bind(server_address)

    # listen for incoming connections
    sock.listen(1)

    # accept a connection
    connection, client_address = sock.accept()

    # receive the data
    data = connection.recv(1024)

    # close the connection
    connection.close()

    # return the received data
    return data

```

This code demonstrates how data can be sent and received using the TCP/IP protocol suite. The `send_data` function takes in the data to be sent, the destination IP address, and the destination port number. It creates a socket object, connects to the destination, and sends the data. The `receive_data` function takes in the port number to listen on. It creates a socket object, binds it to the port, and listens for incoming connections. When a connection is accepted, it receives the data and returns it.

Network devices in Computer Networks

Network devices are components used to connect computers or other electronic devices together so that they can share files or resources like printers or fax machines. Devices used to setup a Local Area Network (LAN) are the most common type of network devices used by the public. A LAN requires a router,

switch, or hub, cabling or radio technology (for wireless networks), and network adapters. Some of the most common network devices are:

- **Router:** A router is a device that forwards data packets between computer networks. It is connected to two or more data lines from different networks and uses routing tables in its operating system to determine the best path for forwarding the packets.
- **Switch:** A switch is a device that filters and forwards packets between LAN segments. Switches operate at the data link layer (layer 2) of the OSI model and therefore support any packet protocol.
- **Hub:** A hub is a device that connects multiple Ethernet devices together and makes them act as a single network segment. Hubs operate at the physical layer (layer 1) of the OSI model and do not examine network packets.
- **Network Interface Card (NIC):** A network interface card (NIC) is a hardware component that connects a computer to a network. It provides a physical connection between the networking cable and the computer's internal bus.
- **Wireless Access Point (WAP):** A wireless access point (WAP) is a device that allows wireless devices to connect to a wired network using Wi-Fi or related standards. The WAP usually connects to a router as a standalone device, but it can also be an integral component of the router itself.
- **Modem:** A modem is a device that modulates an analog carrier signal to encode digital information and demodulates the signal to decode the transmitted information. The goal is to produce a signal that can be transmitted easily and decoded to reproduce the original digital data.
- **Firewall:** A firewall is a network security system that monitors and controls incoming and outgoing network traffic based on predetermined security rules. A firewall typically establishes a barrier between a trusted internal network and untrusted external network, such as the Internet.

Network Components in Computer Networks

There are several key components that make up a computer network:

1. **Nodes:** These are the devices on the network, such as computers, printers, and servers.
2. **Network Interface Cards (NICs):** These are the hardware components that allow nodes to connect to the network.
3. **Cables and Connectors:** These are the physical components that connect the nodes and allow data to be transmitted between them.
4. **Switches and Hubs:** These are devices that connect multiple nodes and direct data traffic between them.

5. **Routers:** These are devices that connect multiple networks and direct data traffic between them.
6. **Network Operating System (NOS):** This is the software that manages the network and allows nodes to communicate with each other.

Physical Layer in Computer Networks

The physical layer is the first layer of the OSI model. It is responsible for the transmission and reception of raw data between a device and a physical transmission medium. This layer is responsible for converting digital data into a signal that can be transmitted over the physical medium, such as electrical signals, light pulses, or radio waves.

Here is an example of a simple physical layer implementation in Python:

```
import serial

class PhysicalLayer:
    def __init__(self, port, baudrate):
        self.ser = serial.Serial(port, baudrate)

    def send(self, data):
        self.ser.write(data)

    def receive(self):
        return self.ser.read()
```

This code creates a `PhysicalLayer` class that uses the `pyserial` library to send and receive data over a serial port. The `send` method takes in data as a parameter and writes it to the serial port. The `receive` method reads data from the serial port and returns it. The `__init__` method takes in the port and baudrate as parameters and initializes the serial connection.

Network topology design in Computer Networks

Network topology refers to the arrangement of the various elements (links, nodes, etc.) of a computer network. There are several different types of network topologies, including star, ring, bus, mesh, and tree. Each topology has its own advantages and disadvantages, and the choice of topology depends on factors such as the size and complexity of the network, the desired level of reliability and performance, and the cost of implementation.

Here is an example of how to design a star topology network in Python:

```
class Network:
    def __init__(self, n):
        self.matrix = []
        self.n = n
```

```

def addlink(self, u, v, w):
    self.matrix.append((u, v, w))

def printMatrix(self):
    for i in range(self.n):
        for j in range(self.n):
            print(self.matrix[i][j], end=" ")
        print("")

def main():
    size = 4
    network = Network(size)
    network.addlink(0, 1, 1)
    network.addlink(0, 2, 1)
    network.addlink(0, 3, 1)
    network.addlink(1, 0, 1)
    network.addlink(1, 2, 1)
    network.addlink(1, 3, 1)
    network.addlink(2, 0, 1)
    network.addlink(2, 1, 1)
    network.addlink(2, 3, 1)
    network.addlink(3, 0, 1)
    network.addlink(3, 1, 1)
    network.addlink(3, 2, 1)
    network.printMatrix()

if __name__ == "__main__":
    main()

```

This code creates a star topology network with 4 nodes. The `addlink` method is used to add links between the nodes, and the `printMatrix` method prints the adjacency matrix of the network. The `main` function creates an instance of the `Network` class and adds links between the nodes to create a star topology. When the `printMatrix` method is called, the adjacency matrix of the network is printed to the console.

Types of connections in Computer Networks

There are two main types of connections in computer networks: **point-to-point** and **multipoint**.

A **point-to-point** connection is a direct link between two devices. This type of connection is typically used for dedicated links between two devices, such as a computer and a printer, or a computer and a server.

A **multipoint** connection, also known as a **broadcast** connection, is a link between multiple devices. This type of connection is typically used in local

area networks (LANs) where multiple devices share a common communication channel.

Both types of connections can use either wired or wireless technologies for data transmission. Wired technologies include Ethernet, while wireless technologies include Wi-Fi and Bluetooth.

Transmission media in Computer Networks

Transmission media refers to the physical pathways that carry data signals between devices in a computer network. There are two main types of transmission media: guided and unguided. Guided media, also known as wired or bounded media, includes copper cables and fiber optic cables. Unguided media, also known as wireless or unbounded media, includes radio waves, microwaves, and infrared waves.

Here is an example of code that can be used to represent the different types of transmission media in a computer network:

```
class TransmissionMedia:
    def __init__(self, media_type, guided):
        self.media_type = media_type
        self.guided = guided

    def __str__(self):
        return f'{self.media_type} is {"guided" if self.guided else "unguided"} media.'

copper_cable = TransmissionMedia('Copper cable', True)
fiber_optic_cable = TransmissionMedia('Fiber optic cable', True)
radio_wave = TransmissionMedia('Radio wave', False)
microwave = TransmissionMedia('Microwave', False)
infrared_wave = TransmissionMedia('Infrared wave', False)

print(copper_cable)
print(fiber_optic_cable)
print(radio_wave)
print(microwave)
print(infrared_wave)
```

This code defines a `TransmissionMedia` class that takes in two arguments: `media_type` and `guided`. The `media_type` argument represents the type of transmission media, while the `guided` argument is a boolean value that indicates whether the media is guided or unguided. The `__str__` method is used to define how the object should be represented as a string. In this case, it returns a string that indicates the media type and whether it is guided or unguided.

The code then creates five instances of the `TransmissionMedia` class, representing copper cable, fiber optic cable, radio wave, microwave, and infrared wave.

These objects are then printed to the console, showing their media type and whether they are guided or unguided.

Signal transmission and encoding in Computer Networks

Signal transmission and encoding are important aspects of computer networks. Signal transmission refers to the process of transmitting data from one point to another, while encoding refers to the process of converting data into a format that can be transmitted over a network.

Here is an example of how signal transmission and encoding might be implemented in a computer network:

```
def transmit_data(data, destination):
    encoded_data = encode_data(data)
    send_data(encoded_data, destination)

def encode_data(data):
    # Implement encoding algorithm here
    encoded_data = ...
    return encoded_data

def send_data(data, destination):
    # Implement data transmission here
    ...
```

This code defines three functions: `transmit_data`, `encode_data`, and `send_data`. The `transmit_data` function takes in data and a destination as arguments, and first encodes the data using the `encode_data` function. The encoded data is then sent to the destination using the `send_data` function.

The `encode_data` function is where the actual encoding of the data takes place. This function would need to be implemented with a specific encoding algorithm, depending on the requirements of the network.

The `send_data` function is responsible for actually transmitting the data to the destination. This function would need to be implemented with the specific details of how data is transmitted in the network, such as the protocol used and the details of the physical transmission medium.

Overall, signal transmission and encoding are crucial for ensuring that data can be reliably transmitted over a computer network. By properly encoding the data and transmitting it using a well-designed protocol, a computer network can efficiently and reliably transmit data between different points.

Network performance and transmission impairments in Computer Networks

Network performance refers to the level of quality of service of a telecommunications product as seen by the customer. It should not be seen merely as an attempt to get “more through” the network. Performance can also be modeled and predicted instead of measured; one example of this is using state transition diagrams to model queuing performance or to use a Network Simulator.

Transmission impairments are any factors that negatively affect the quality of the transmitted signal. These impairments can include attenuation, distortion, noise, and interference. Attenuation is the loss of signal strength as it travels through the transmission medium. Distortion is the alteration of the original signal, which can be caused by a variety of factors, including the characteristics of the transmission medium and the presence of other signals. Noise is any unwanted signal that interferes with the transmission of the desired signal. Interference is the presence of other signals that can disrupt the transmission of the desired signal.

Here is an example of a Python code that calculates the network performance based on the transmission impairments:

```
def network_performance(attenuation, distortion, noise, interference):
    performance = 100 - (attenuation + distortion + noise + interference)
    return performance

attenuation = 10
distortion = 5
noise = 15
interference = 20

performance = network_performance(attenuation, distortion, noise, interference)
print("Network performance:", performance)
```

Switching techniques and multiplexing in Computer Networks

Switching techniques and multiplexing are two important concepts in computer networks. Switching techniques refer to the methods used to transfer data between different devices in a network. Multiplexing, on the other hand, refers to the process of combining multiple signals into a single signal for transmission over a shared medium.

There are three main switching techniques used in computer networks: circuit switching, packet switching, and message switching.

Circuit switching involves establishing a dedicated communication path between two devices for the duration of their communication. This path is reserved exclusively for the use of these two devices, and no other devices can use it until the communication is complete.

Packet switching, on the other hand, involves breaking data into small packets and sending them individually over the network. Each packet is routed inde-

pendently, and may take a different path to its destination. This allows for more efficient use of network resources, as multiple devices can share the same communication path.

Message switching is similar to packet switching, but instead of breaking data into small packets, entire messages are sent as a single unit. Each message is stored at intermediate nodes until a suitable communication path is available, and then forwarded to the next node.

Multiplexing can be achieved using either time-division multiplexing (TDM) or frequency-division multiplexing (FDM). TDM involves dividing the available transmission time into time slots, and assigning each signal to a different time slot. FDM, on the other hand, involves dividing the available frequency spectrum into frequency bands, and assigning each signal to a different frequency band.

Unit 2 - Link layer in Computer Networks and Medium Access Control and Local Area Networks

The link layer is the lowest layer in the OSI model of computer networking. It is responsible for the transmission of data between two directly connected nodes. The link layer is responsible for providing services such as framing, error detection and correction, and flow control.

Medium Access Control (MAC) is a sublayer of the link layer that controls how devices in a network access the shared communication medium. MAC protocols are used to ensure that only one device transmits on the medium at a time, to avoid collisions.

Local Area Networks (LANs) are computer networks that are designed to operate over a small geographical area, such as a home, office, or campus. LANs typically use a shared communication medium, such as Ethernet or Wi-Fi, and are characterized by high data transfer rates and low latency.

Here is an example of a simple MAC protocol, called the Aloha protocol:

```
def aloha(time, frame):  
    if time % 2 == 0:  
        transmit(frame)  
    else:  
        wait()
```

This protocol allows devices to transmit frames at even time slots, and waits at odd time slots. This reduces the chance of collisions, but can result in low channel utilization if there are many devices in the network.

Link layer in Computer Networks

The link layer is the lowest layer in the OSI model of computer networking. It is

responsible for the transmission of data between two directly connected nodes. This layer is responsible for the physical addressing of the data, error detection and correction, and flow control.

Here is an example of a simple link layer protocol written in C:

```
#include <stdio.h>
#include <string.h>

#define MAX_PACKET_SIZE 1024

typedef struct {
    char dest[6];
    char src[6];
    unsigned short type;
    char data[MAX_PACKET_SIZE];
    unsigned int crc;
} packet_t;

void send_packet(packet_t *packet) {
    // implementation of sending packet over the physical link
}

void receive_packet(packet_t *packet) {
    // implementation of receiving packet from the physical link
}

int main() {
    packet_t packet;
    strcpy(packet.dest, "00:11:22:33:44:55");
    strcpy(packet.src, "AA:BB:CC:DD:EE:FF");
    packet.type = 0x0800; // IPv4
    strcpy(packet.data, "Hello, World!");
    packet.crc = 0; // calculate CRC

    send_packet(&packet);

    return 0;
}
```

Framing in link layer in Computer Networks

Framing is the process of encapsulating data into a frame at the link layer of the OSI model. A frame is a unit of data transmission that includes not only the data itself, but also additional information such as the source and destination addresses, error detection and correction information, and control information.

Here is an example of how framing can be implemented in C:

```
#include <stdio.h>
#include <string.h>

#define MAX_FRAME_SIZE 1024

typedef struct {
    char source[6];
    char destination[6];
    char data[MAX_FRAME_SIZE];
    int data_size;
    char checksum[4];
} frame_t;

void create_frame(frame_t *frame, char *source, char *destination, char *data, int data_size) {
    memcpy(frame->source, source, 6);
    memcpy(frame->destination, destination, 6);
    memcpy(frame->data, data, data_size);
    frame->data_size = data_size;
    // Calculate checksum here
}

void print_frame(frame_t *frame) {
    printf("Source: %s\n", frame->source);
    printf("Destination: %s\n", frame->destination);
    printf("Data: %s\n", frame->data);
    printf("Data size: %d\n", frame->data_size);
    printf("Checksum: %s\n", frame->checksum);
}

int main() {
    frame_t frame;
    create_frame(&frame, "Alice", "Bob", "Hello, Bob!", strlen("Hello, Bob!"));
    print_frame(&frame);
    return 0;
}
```

This code defines a `frame_t` structure that represents a frame, with fields for the source and destination addresses, the data, the data size, and the checksum. The `create_frame` function takes a pointer to a `frame_t` structure, the source and destination addresses, the data, and the data size as arguments, and initializes the frame with the given values. The `print_frame` function takes a pointer to a `frame_t` structure as an argument and prints the contents of the frame.

Error Detection and Correction in link layer in Computer Networks

Error detection and correction are techniques used in the link layer of computer networks to ensure that data is transmitted accurately. These techniques are used to detect and correct errors that may occur during transmission due to noise, interference, or other factors.

One common method for error detection is the use of parity bits. A parity bit is an extra bit added to a data unit that indicates whether the number of 1s in the data unit is even or odd. If an error occurs during transmission and the number of 1s changes, the parity bit will no longer match the data, indicating that an error has occurred.

Another common method for error detection is the use of checksums. A checksum is a value calculated from the data in a data unit. The sender calculates the checksum and includes it with the data when it is transmitted. The receiver recalculates the checksum from the received data and compares it to the checksum received with the data. If the two values do not match, an error has occurred.

Error correction is more complex than error detection and involves not only detecting errors but also correcting them. One common method for error correction is the use of Hamming codes. Hamming codes are a type of error-correcting code that can detect and correct single-bit errors. They work by adding extra bits to the data, called check bits, that are used to detect and correct errors.

These are just a few examples of the many techniques used for error detection and correction in the link layer of computer networks. These techniques are essential for ensuring the reliable transmission of data over noisy and unreliable communication channels.

Flow control in link layer in Computer Networks

Flow control is a mechanism used in the link layer of computer networks to prevent the sender from overwhelming the receiver with data. This is achieved by regulating the rate at which data is transmitted from the sender to the receiver.

Here is an example of flow control using the sliding window protocol in the link layer:

```
# Sender
window_size = 4
next_frame_to_send = 0
max_seq = 2 * window_size

def send_data(data):
    global next_frame_to_send
```

```

    while data:
        while next_frame_to_send < window_size:
            send_frame(data.pop(0), next_frame_to_send)
            next_frame_to_send = (next_frame_to_send + 1) % max_seq

def send_frame(frame, seq_num):
    # send frame with sequence number seq_num
    pass

def receive_ack(ack_num):
    global next_frame_to_send
    next_frame_to_send = max(next_frame_to_send, ack_num + 1)

# Receiver
expected_frame = 0

def receive_frame(frame, seq_num):
    global expected_frame
    if seq_num == expected_frame:
        deliver_data(frame)
        expected_frame = (expected_frame + 1) % max_seq
        send_ack(expected_frame)

def send_ack(ack_num):
    # send acknowledgement with ack_num
    pass

def deliver_data(data):
    # deliver data to the upper layer
    pass

```

This code demonstrates how the sender and receiver use the sliding window protocol to regulate the flow of data between them. The sender maintains a window of frames that it is allowed to send, and the receiver sends acknowledgements to the sender to indicate which frames have been received. The sender adjusts its window based on the acknowledgements received from the receiver, ensuring that the receiver is not overwhelmed with data.

Elementary Data Link Protocols in link layer in Computer Networks

Elementary Data Link Protocols are the protocols used in the link layer of computer networks to provide reliable communication between two adjacent nodes. These protocols are responsible for framing, flow control, and error control.

Here is an example of a simple Stop-and-Wait protocol implemented in Python:

```

def stop_and_wait(sender, receiver, data):
    # sender sends data to receiver using stop-and-wait protocol
    for packet in data:
        # sender sends packet
        sender.send(packet)
        # wait for acknowledgment from receiver
        ack = receiver.receive()
        # if acknowledgment is not received, resend packet
        while ack != packet:
            sender.send(packet)
            ack = receiver.receive()

```

This protocol works by having the sender send a packet of data and then wait for an acknowledgment from the receiver before sending the next packet. If the acknowledgment is not received, the sender resends the packet until it is successfully received by the receiver.

This is just one example of an elementary data link protocol. There are many other protocols that can be used in the link layer of computer networks to provide reliable communication between adjacent nodes.

Sliding Window protocols in link layer in Computer Networks

Sliding Window protocols are used in the link layer of computer networks to provide reliable data transfer. These protocols use a window of sequence numbers to keep track of the frames that have been sent and acknowledged. Here is an example of a sliding window protocol implemented in Python:

```

# Sender
window_size = 4
next_seq_num = 0
base = 0

def send_data(data):
    global next_seq_num
    global base
    while base < len(data):
        while next_seq_num < base + window_size and next_seq_num < len(data):
            send_frame(data[next_seq_num], next_seq_num)
            next_seq_num += 1
        receive_ack()

def send_frame(frame, seq_num):
    # send the frame with the sequence number
    pass

def receive_ack():

```

```

    global base
    # receive the acknowledgement and update the base
    pass

# Receiver
expected_seq_num = 0

def receive_data():
    global expected_seq_num
    while True:
        frame, seq_num = receive_frame()
        if seq_num == expected_seq_num:
            deliver_data(frame)
            expected_seq_num += 1
            send_ack(expected_seq_num)

def receive_frame():
    # receive the frame and the sequence number
    pass

def deliver_data(frame):
    # deliver the data to the upper layer
    pass

def send_ack(ack_num):
    # send the acknowledgement
    pass

```

This is a basic implementation of a sliding window protocol. The sender keeps track of the next sequence number to be sent and the base of the window. The receiver keeps track of the expected sequence number. When the receiver receives a frame with the expected sequence number, it delivers the data to the upper layer and sends an acknowledgement. The sender updates the base of the window when it receives an acknowledgement. The window size can be adjusted to control the flow of data.

Medium Access Control and Local Area Networks

Medium Access Control (MAC) is a sublayer of the Data Link Layer in the OSI model. It is responsible for controlling how devices in a network gain access to a shared medium and transmit data. In a Local Area Network (LAN), the MAC layer is responsible for ensuring that data is transmitted without collisions or interference.

Here is an example of a simple MAC protocol, called the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol, used in Ethernet networks:

```
def csma_cd():
    while True:
        if medium_is_idle():
            transmit_data()
            if collision_detected():
                wait_for_random_time()
            else:
                break
        else:
            wait_for_medium_to_become_idle()
```

This protocol works by first checking if the medium is idle before attempting to transmit data. If the medium is not idle, the device waits for it to become idle before attempting to transmit. If a collision is detected during transmission, the device waits for a random amount of time before attempting to retransmit the data. This process continues until the data is successfully transmitted without collision.

Channel allocation in medium access control

Channel allocation in medium access control refers to the process of assigning communication channels to devices in a network. There are several methods for channel allocation, including fixed, dynamic, and hybrid allocation.

Here is an example of a simple channel allocation algorithm in Python:

```
def channel_allocation(devices, channels):
    allocation = {}
    for device in devices:
        allocation[device] = channels.pop(0)
        channels.append(allocation[device])
    return allocation
```

This algorithm takes a list of devices and a list of available channels as input and returns a dictionary where the keys are the devices and the values are the assigned channels. The algorithm simply assigns the first available channel to each device in the order they appear in the input list. Once a channel is assigned, it is moved to the end of the list of available channels to ensure that it is not assigned again until all other channels have been used.

Multiple access protocols in medium access control

Multiple access protocols are used in medium access control to coordinate access to a shared communication medium. Some common multiple access protocols include:

- **TDMA (Time Division Multiple Access):** This protocol divides the available time into time slots and assigns each slot to a different user. Each user can transmit data only during their assigned time slot.

- **FDMA (Frequency Division Multiple Access):** This protocol divides the available frequency band into smaller sub-bands and assigns each sub-band to a different user. Each user can transmit data only on their assigned frequency.
- **CDMA (Code Division Multiple Access):** This protocol assigns a unique code to each user. Each user can transmit data at any time, but their data is spread over the entire frequency band using their unique code. The receiver can recover the data of a specific user by using the corresponding code.
- **CSMA (Carrier Sense Multiple Access):** This protocol allows users to transmit data only if the channel is sensed to be idle. If the channel is busy, the user must wait until it becomes idle before transmitting.
- **ALOHA:** This protocol allows users to transmit data at any time. If a collision occurs (i.e., two or more users transmit data at the same time), the users must retransmit their data after a random delay.

These are some of the common multiple access protocols used in medium access control. Each protocol has its own advantages and disadvantages, and the choice of protocol depends on the specific requirements of the communication system.

LAN standards in local area network

There are several LAN standards that have been developed over the years to meet the needs of local area networks. Some of the most common LAN standards include:

- **Ethernet:** Ethernet is the most widely used LAN technology. It uses a bus or star topology and supports data transfer rates of 10 Mbps, 100 Mbps, 1 Gbps, and 10 Gbps.
- **Wi-Fi:** Wi-Fi is a wireless LAN technology that uses radio waves to transmit data. It supports data transfer rates of up to 1.3 Gbps and is commonly used in homes, offices, and public places.
- **Token Ring:** Token Ring is a LAN technology that uses a ring topology. It supports data transfer rates of 4 Mbps and 16 Mbps.
- **FDDI:** FDDI (Fiber Distributed Data Interface) is a LAN technology that uses fiber optic cables to transmit data. It supports data transfer rates of up to 100 Mbps.

Link layer switches & bridges in local area network

Link layer switches and bridges are used in local area networks (LANs) to connect multiple devices and facilitate communication between them. These devices operate at the link layer (layer 2) of the OSI model and are responsible for forwarding data frames between devices on the same network.

Here is an example of how a link layer switch can be implemented in a local area network using the C programming language:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_DEVICES 10
#define MAX_FRAME_SIZE 1500

typedef struct {
    char src[6];
    char dst[6];
    char data[MAX_FRAME_SIZE];
} frame_t;

typedef struct {
    char mac[6];
    int port;
} device_t;

device_t devices[MAX_DEVICES];
int device_count = 0;

void add_device(char *mac, int port) {
    if (device_count >= MAX_DEVICES) {
        printf("Error: Maximum number of devices reached\n");
        return;
    }
    memcpy(devices[device_count].mac, mac, 6);
    devices[device_count].port = port;
    device_count++;
}

int get_device_port(char *mac) {
    for (int i = 0; i < device_count; i++) {
        if (memcmp(devices[i].mac, mac, 6) == 0) {
            return devices[i].port;
        }
    }
    return -1;
}

void forward_frame(frame_t *frame) {
    int dst_port = get_device_port(frame->dst);
    if (dst_port == -1) {
```



```

        printf("Error: Destination device not found\n");
        return;
    }
    printf("Forwarding frame from port %d to port %d\n", get_device_port(frame->src), dst_port);
}

int main() {
    add_device("\x00\x11\x22\x33\x44\x55", 1);
    add_device("\x66\x77\x88\x99\xaa\xbb", 2);

    frame_t frame;
    memcpy(frame.src, "\x00\x11\x22\x33\x44\x55", 6);
    memcpy(frame.dst, "\x66\x77\x88\x99\xaa\xbb", 6);
    strcpy(frame.data, "Hello, world!");

    forward_frame(&frame);

    return 0;
}

```

This code defines a `frame_t` structure to represent a data frame, and a `device_t` structure to represent a device on the network. The `add_device` function is used to add devices to the network, and the `get_device_port` function is used to find the port number of a device based on its MAC address. The `forward_frame` function is responsible for forwarding a frame from its source to its destination by looking up the destination device's port number and forwarding the frame accordingly.

This is just one example of how link layer switches and bridges can be implemented in a local area network. There are many other ways to implement these devices, and the specific implementation will depend on the requirements of the network and the devices being used.

Learning bridge algorithms in local area network

A bridge algorithm is used in a local area network (LAN) to connect two or more network segments and regulate the flow of data between them. Bridges operate at the data link layer (layer 2) of the OSI model and use MAC addresses to determine where to forward traffic.

Here is an example of a simple bridge algorithm in Python:

```

def bridge_algorithm(mac_table, incoming_frame, incoming_port):
    source_mac, destination_mac = incoming_frame[0], incoming_frame[1]
    mac_table[source_mac] = incoming_port
    if destination_mac in mac_table:
        outgoing_port = mac_table[destination_mac]
        if outgoing_port != incoming_port:

```

```

        return outgoing_port
    return None

```

This algorithm takes as input a MAC address table, an incoming frame, and the port on which the frame was received. The algorithm first updates the MAC address table with the source MAC address and the incoming port. Then, it checks if the destination MAC address is in the MAC address table. If it is, the algorithm returns the outgoing port associated with the destination MAC address, as long as it is not the same as the incoming port. If the destination MAC address is not in the MAC address table, the algorithm returns `None`, indicating that the frame should be flooded to all ports.

Spanning Tree Algorithms in Local Area Network

- Spanning Tree Protocol (STP) is a network protocol that builds a loop-free logical topology for Ethernet networks.
- The basic function of STP is to prevent bridge loops and the broadcast radiation that results from them.
- Spanning tree also allows a network design to include backup links providing fault tolerance if an active link fails.
- The spanning-tree algorithm blocks forwarding on redundant links by setting up one preferred link between switches in the LAN.
- This preferred link is used for all Ethernet frames unless it fails, in which case a non-preferred redundant link is enabled.
- When implemented in a network, STP designates one layer-2 switch as root bridge.
- The spanning tree algorithm allows to remove logical rings from the network physical topology, by disabling links to transform a mesh topology (graph) into a tree called spanning tree, whose root is one of the bridges called root bridge.

Unit 3 - Network Layer in Computer Networks

The network layer is responsible for routing data packets from the source to the destination. This layer is responsible for logical addressing and routing. The network layer uses routing algorithms to determine the best path for data packets to take from the source to the destination.

Here is an example of a simple routing algorithm in Python:

```

def route_packet(source, destination, network_graph):
    # Use a shortest path algorithm to find the best path
    best_path = shortest_path(network_graph, source, destination)
    # Forward the packet along the best path
    forward_packet(packet, best_path)

```

This code uses a shortest path algorithm to determine the best path for a packet to take from the source to the destination. The packet is then forwarded along

this path. This is just one example of how routing can be implemented at the network layer. There are many other routing algorithms and techniques that can be used.

Point-to-point networks in network layer

A point-to-point network is a type of network topology in which each pair of nodes is connected by a dedicated communication link. In the network layer, point-to-point networks are used to establish a direct connection between two devices, allowing them to communicate with each other without the need for intermediate devices.

Here is an example of how a point-to-point network can be implemented in the network layer using the Python programming language:

```
import socket

# Create a socket object
s = socket.socket()

# Define the port on which you want to connect
port = 12345

# Connect to the server on the local computer
s.connect(('127.0.0.1', port))

# Send a message to the server
s.sendall(b'Hello, server!')

# Receive data from the server
data = s.recv(1024)

# Print the received data
print(data)

# Close the socket
s.close()
```

This code creates a socket object and uses it to establish a connection to a server running on the local computer. Once the connection is established, the client can send and receive data from the server using the `sendall` and `recv` methods, respectively. Finally, the socket is closed to release the resources associated with the connection.

Logical addressing in network layer

Logical addressing is used in the network layer to identify devices on a network and to route data between them. The most common logical addressing scheme

is the Internet Protocol (IP) addressing, which uses a 32-bit (IPv4) or 128-bit (IPv6) address to uniquely identify devices on a network.

Here is an example of how logical addressing is used in the network layer:

```
import socket

# create a socket object
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# get local machine name
host = socket.gethostname()

# connection to hostname on the port
s.connect((host, 80))

# receive data from the server
data = s.recv(1024)

s.close()
print(data)
```

Basic internetworking in network layer

Internetworking is the process of connecting multiple computer networks together to form a larger network. The network layer is responsible for providing logical addressing and routing services in an internetwork. Here is an example of how basic internetworking can be implemented in the network layer using the Internet Protocol (IP):

```
import socket

# Create a socket object
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Get local machine name
host = socket.gethostname()

# Reserve a port for your service
port = 12345

# Bind the socket to the port
s.bind((host, port))

# Become a server socket
s.listen(5)
```

```

while True:
    # Establish a connection
    clientsocket, addr = s.accept()
    print("Got a connection from %s" % str(addr))
    clientsocket.send("Thank you for connecting")
    clientsocket.close()

```

This code creates a socket object, binds it to a port, and listens for incoming connections. When a connection is established, the server sends a message to the client and closes the connection. This is a basic example of how internetworking can be implemented in the network layer using the IP protocol.

IP

```

import socket

def get_ip():
    hostname = socket.gethostname()
    ip_address = socket.gethostbyname(hostname)
    return ip_address

print(get_ip())

```

CIDR

CIDR (Classless Inter-Domain Routing) is a way to represent IP addresses and their associated network masks. Here is an example of CIDR notation in Python:

```

import ipaddress

# Define the IP address and network mask in CIDR notation
cidr = '192.168.1.0/24'

# Create an IPv4 network object
network = ipaddress.ip_network(cidr)

# Print the network address and network mask
print(f'Network address: {network.network_address}')
print(f'Network mask: {network.netmask}')

```

ARP

Here is an example of an Address Resolution Protocol (ARP) code in Python:

```

import socket
import struct
import binascii

```

```

s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.ntohs(0x0003))
while True:
    packet = s.recvfrom(2048)
    ethernet_header = packet[0][0:14]
    ethernet_detailed = struct.unpack("!6s6s2s", ethernet_header)
    arp_header = packet[0][14:42]
    arp_detailed = struct.unpack("2s2s1s1s2s6s4s6s4s", arp_header)
    # skip non-ARP packets
    ethertype = ethernet_detailed[2]
    if ethertype != b'\x08\x06':
        continue
    source_mac = binascii.hexlify(ethernet_detailed[1]).decode('ascii')
    dest_mac = binascii.hexlify(ethernet_detailed[0]).decode('ascii')
    source_ip = socket.inet_ntoa(arp_detailed[6])
    dest_ip = socket.inet_ntoa(arp_detailed[8])
    print("Ethernet Frame: ")
    print("Source MAC: {}, Destination MAC: {}".format(source_mac, dest_mac))
    print("ARP Packet: ")
    print("Source IP: {}, Destination IP: {}".format(source_ip, dest_ip))

```

This code listens for incoming ARP packets and prints the source and destination MAC and IP addresses of the Ethernet frame and ARP packet, respectively.

RARP

RARP (Reverse Address Resolution Protocol) is a protocol used to resolve an IP address from a given hardware address (such as an Ethernet address). Here is an example of RARP code in C:

```

#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <net/if_arp.h>
#include <string.h>

int main() {
    int s;
    struct arpreq arpreq;
    struct sockaddr_in *sin;
    unsigned char *eap;

    s = socket(AF_INET, SOCK_DGRAM, 0);
    if (s == -1) {
        perror("socket");
        return 1;
    }
}

```

```

memset(&arpreq, 0, sizeof(arpreq));
sin = (struct sockaddr_in *) &arpreq.arp_pa;
sin->sin_family = AF_INET;
sin->sin_addr.s_addr = inet_addr("192.168.1.1");
strcpy(arpreq.arp_dev, "eth0");

if (ioctl(s, SIOCGARP, &arpreq) == -1) {
    perror("ioctl");
    return 1;
}

eap = (unsigned char *) &arpreq.arp_ha.sa_data[0];
printf("HW address is %02X:%02X:%02X:%02X:%02X:%02X\n",
       eap[0], eap[1], eap[2], eap[3], eap[4], eap[5]);

return 0;
}

```

This code creates a socket and uses the `ioctl` function to send a `SIOCGARP` request to the kernel to retrieve the hardware address associated with the given IP address (192.168.1.1 in this example). The hardware address is then printed in the standard format.

DHCP

Dynamic Host Configuration Protocol (DHCP) is a network management protocol used to dynamically assign an Internet Protocol (IP) address to any device, or node, on a network so they can communicate using IP. Here is an example of a simple DHCP server written in Python:

```

import socket
from struct import pack

server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind(('', 67))

while True:
    message, address = server_socket.recvfrom(1024)
    transaction_id = message[4:8]
    client_mac = message[28:34]
    offer_ip = socket.inet_aton('192.168.1.100')
    server_ip = socket.inet_aton('192.168.1.1')
    subnet_mask = socket.inet_aton('255.255.255.0')
    router = socket.inet_aton('192.168.1.1')
    lease_time = pack('!L', 86400)
    dhcp_message_type = b'\x35\x01\x02'

```

```

end_option = b'\xff'

offer = b'\x02' + transaction_id + b'\x00\x00\x00\x00' + offer_ip + server_ip + b'\x00'
server_socket.sendto(offer, ('<broadcast>', 68))

```

This code creates a DHCP server that listens on port 67 for DHCPDISCOVER messages from clients. When it receives a message, it extracts the transaction ID and client MAC address from the message and uses them to create a DHCPOFFER message offering an IP address to the client. The server then broadcasts the offer to the client on port 68.

ICMP

- ICMP stands for **Internet Control Message Protocol**.
- It is a protocol that devices within a network use to communicate problems with data transmission.
- One of the primary ways in which ICMP is used is to determine if data is getting to its destination and at the right time.
- ICMP is used for reporting errors and performing network diagnostics.
- In the error reporting process, ICMP sends messages from the receiver to the sender when data does not come through as it should.
- ICMP is part of the Internet protocol suite as defined in RFC 792.
- ICMP messages are typically used for diagnostic or control purposes or generated in response to errors in IP operations (as specified in RFC 1122).
- ICMP errors are directed to the source IP address of the originating packet.
- ICMP is a network layer protocol used by network devices to diagnose network communication issues.
- Commonly, the ICMP protocol is used on network devices, such as routers.

Routing in network layer

Routing is the process of selecting a path for traffic in a network or between or across multiple networks. The network layer is responsible for routing packets from the source to the destination. Here is an example of a routing algorithm in Python:

```

def dijkstra(graph, start, end):
    shortest_paths = {start: (None, 0)}
    current_node = start
    visited = set()

    while current_node != end:
        visited.add(current_node)
        destinations = graph.edges[current_node]
        weight_to_current_node = shortest_paths[current_node][1]

        for next_node in destinations:

```



```

weight = graph.weights[(current_node, next_node)] + weight_to_current_node
if next_node not in shortest_paths:
    shortest_paths[next_node] = (current_node, weight)
else:
    current_shortest_weight = shortest_paths[next_node][1]
    if current_shortest_weight > weight:
        shortest_paths[next_node] = (current_node, weight)

next_destinations = {node: shortest_paths[node] for node in shortest_paths if node != current_node}
if not next_destinations:
    return "Route Not Possible"
current_node = min(next_destinations, key=lambda k: next_destinations[k][1])

path = []
while current_node is not None:
    path.append(current_node)
    next_node = shortest_paths[current_node][0]
    current_node = next_node
path = path[::-1]
return path

```

This is an implementation of Dijkstra's algorithm, which is used to find the shortest path between nodes in a graph. It can be used for routing in a network by representing the network as a graph, where nodes are devices and edges are connections between them. The weights on the edges represent the cost of transmitting data along that connection, such as the distance or the amount of traffic. The algorithm finds the path with the lowest total cost from the source to the destination.

Forwarding and Delivery in Network Layer

Forwarding refers to the process of moving a packet from an incoming link to an outgoing link within a router. Delivery refers to the process of moving a packet from the source host to the destination host.

Here is an example of how forwarding and delivery might be implemented in a network layer:

```

def forward_packet(packet, incoming_link, outgoing_link):
    # code to move packet from incoming_link to outgoing_link
    pass

def deliver_packet(packet, source_host, destination_host):
    # code to move packet from source_host to destination_host
    pass

```

Static and dynamic routing in cn

Static routing is a type of routing that is manually configured by a network administrator. The administrator creates a routing table that specifies the path for each destination network. This type of routing is suitable for small networks, where the network topology is simple and does not change frequently.

On the other hand, dynamic routing is a type of routing that is automatically configured by routing protocols. The routing protocols exchange information about the network topology and calculate the best path for each destination network. This type of routing is suitable for large networks, where the network topology is complex and changes frequently.

Here is an example of how to configure static routing on a Cisco router:

```
Router(config)# ip route [destination network] [subnet mask] [next-hop address or exit inter
```

And here is an example of how to configure dynamic routing using the OSPF routing protocol on a Cisco router:

```
Router(config)# router ospf [process ID]
Router(config-router)# network [network address] [wildcard mask] area [area ID]
```

Routing algorithms and protocols in cn

Routing algorithms and protocols are used in computer networks to determine the best path for data to travel from one device to another. There are several types of routing algorithms, including distance-vector, link-state, and path-vector. Each algorithm has its own set of rules and procedures for calculating the best path.

Distance-vector algorithms use information about the distance and direction to a destination to determine the best path. This information is shared between neighboring routers, allowing them to update their routing tables and make better routing decisions.

Link-state algorithms, on the other hand, use information about the entire network topology to determine the best path. Each router maintains a map of the network and uses a shortest-path algorithm to find the best route to a destination.

Path-vector algorithms are similar to distance-vector algorithms, but they also include information about the path taken to reach a destination. This allows routers to avoid routing loops and make more informed routing decisions.

Some common routing protocols used in computer networks include the Routing Information Protocol (RIP), Open Shortest Path First (OSPF), and Border Gateway Protocol (BGP). These protocols use different routing algorithms and have different strengths and weaknesses, making them suitable for different types of networks.

In summary, routing algorithms and protocols are essential components of computer networks, allowing data to be efficiently transmitted from one device to another. Different algorithms and protocols are used depending on the specific needs and characteristics of the network.

Congestion control algorithms in cn

Congestion control algorithms are used in computer networks to prevent congestion and ensure efficient data transmission. Some common congestion control algorithms include:

- **Additive Increase Multiplicative Decrease (AIMD):** This algorithm increases the congestion window size by one maximum segment size (MSS) for each round trip time (RTT) until a packet loss is detected. When a packet loss is detected, the congestion window size is reduced by half.
- **Slow Start:** This algorithm starts with a small congestion window size and increases it exponentially until a threshold is reached. After the threshold is reached, the congestion window size is increased linearly.
- **Congestion Avoidance:** This algorithm is used to avoid congestion by increasing the congestion window size slowly and steadily. It uses a combination of slow start and additive increase to increase the congestion window size.
- **Fast Retransmit:** This algorithm is used to quickly retransmit lost packets. When a packet loss is detected, the sender immediately retransmits the lost packet without waiting for the retransmission timer to expire.
- **Fast Recovery:** This algorithm is used to recover from packet loss quickly. When a packet loss is detected, the sender reduces the congestion window size by half and enters the fast recovery phase. In the fast recovery phase, the sender increases the congestion window size by one MSS for each duplicate acknowledgment received.

These are some of the common congestion control algorithms used in computer networks. They help to prevent congestion and ensure efficient data transmission.

IPv6 in CN

Here is an example of how to create an IPv6 socket in Python:

```
import socket

# Create an IPv6 socket
sock = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)
```

```

# Bind the socket to a specific address and port
sock.bind(('::1', 12345))

# Listen for incoming connections
sock.listen(1)

# Accept a connection
conn, addr = sock.accept()

# Receive data from the client
data = conn.recv(1024)

# Send data back to the client
conn.sendall(data)

# Close the connection
conn.close()

```

Unit 4 - Transport Layer in Computer Networks

The transport layer is responsible for providing end-to-end communication between applications on different devices in a computer network. This layer is responsible for establishing, maintaining, and terminating connections between applications, as well as ensuring reliable data transfer and flow control.

Here is an example of a simple transport layer protocol implemented in Python:

```

import socket

def transport_layer_send(data, destination_ip, destination_port):
    # create a socket object
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # connect to the destination
    sock.connect((destination_ip, destination_port))

    # send the data
    sock.sendall(data)

    # close the connection
    sock.close()

def transport_layer_receive(port):
    # create a socket object
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # bind the socket to the port

```

```

server_address = ('localhost', port)
sock.bind(server_address)

# listen for incoming connections
sock.listen(1)

# accept a connection
connection, client_address = sock.accept()

# receive the data
data = connection.recv(1024)

# close the connection
connection.close()

return data

```

Process-to-process delivery in transport layer

The transport layer is responsible for providing process-to-process delivery of data between applications running on different hosts. This is achieved through the use of transport layer protocols such as TCP (Transmission Control Protocol) and UDP (User Datagram Protocol).

Here is an example of how process-to-process delivery works in the transport layer using the TCP protocol:

```

# Client-side code
from socket import socket, AF_INET, SOCK_STREAM

serverName = 'hostname'
serverPort = 12000

clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))

sentence = input('Input lowercase sentence: ')
clientSocket.send(sentence.encode())

modifiedSentence = clientSocket.recv(1024)
print('From Server: ', modifiedSentence.decode())

clientSocket.close()

# Server-side code
from socket import socket, AF_INET, SOCK_STREAM

```

```

serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind('', serverPort)
serverSocket.listen(1)

print('The server is ready to receive')

while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()

```

In this example, the client-side code creates a TCP socket and connects to the server using the server's hostname and port number. The client then sends a sentence to the server, which is received and processed by the server-side code. The server capitalizes the sentence and sends it back to the client, where it is received and printed. This is an example of process-to-process delivery, where data is sent from one process (the client) to another process (the server) using the transport layer.

Transport layer protocols

The transport layer is responsible for providing end-to-end communication services for applications. It provides services such as connection-oriented data stream support, reliability, flow control, and multiplexing. The two most common transport layer protocols are the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP).

Here is an example of how to create a TCP socket in Python:

```

import socket

# create a TCP/IP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# connect the socket to the server
server_address = ('localhost', 10000)
sock.connect(server_address)

```

And here is an example of how to create a UDP socket in Python:

```

import socket

# create a UDP/IP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

```

```
# bind the socket to the port
server_address = ('localhost', 10000)
sock.bind(server_address)
```

UDP Transport layer protocol

Here is an example of a simple UDP server written in Python:

```
import socket

# Create a UDP socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Bind the socket to a specific address and port
server_address = ('localhost', 10000)
server_socket.bind(server_address)

while True:
    # Receive data from the client
    data, address = server_socket.recvfrom(4096)

    # Process the data and send a response back to the client
    response = data.upper()
    server_socket.sendto(response, address)
```

This code creates a UDP socket and binds it to a specific address and port. It then enters an infinite loop where it waits to receive data from a client. Once data is received, it is processed (in this case, by converting it to uppercase) and a response is sent back to the client.

TCP Transport layer protocol

Transmission Control Protocol (TCP) is a standard that defines how to establish and maintain a network conversation through which application programs can exchange data. TCP works with the Internet Protocol (IP), which defines how computers send packets of data to each other. Together, TCP and IP are the basic rules defining the Internet.

Here is an example of a simple TCP server written in Python:

```
import socket

def start_server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(('localhost', 8080))
    server_socket.listen(1)
    print('Server is listening on port 8080')
```

```

while True:
    client_socket, client_address = server_socket.accept()
    print(f'Accepted connection from {client_address}')
    client_socket.sendall(b'Hello, client!')
    client_socket.close()

if __name__ == '__main__':
    start_server()

```

Multiplexing in transport layer

Multiplexing in the transport layer refers to the process of combining multiple data streams from different applications into a single data stream for transmission over the network. This is achieved by assigning a unique identifier, known as a port number, to each application. The transport layer protocol, such as TCP or UDP, uses these port numbers to distinguish between different data streams and ensure that the data is delivered to the correct application on the receiving end.

Here is an example of how multiplexing works in the transport layer using the TCP protocol:

```

import socket

# create a TCP/IP socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# bind the socket to a specific address and port
server_address = ('localhost', 10000)
server_socket.bind(server_address)

# listen for incoming connections
server_socket.listen(1)

while True:
    # wait for a connection
    connection, client_address = server_socket.accept()

    # receive and process data from the client
    data = connection.recv(1024)
    # process data...

    # send response back to the client
    connection.sendall(response)

    # clean up the connection
    connection.close()

```


In this example, the server creates a TCP/IP socket and binds it to a specific address and port (in this case, `localhost` and port 10000). The server then listens for incoming connections on this port. When a client connects to the server, the server accepts the connection and receives data from the client. The server processes the data and sends a response back to the client before closing the connection. This process is repeated for each incoming connection, allowing the server to handle multiple clients simultaneously.

Connection management in transport layer

Connection management in the transport layer is responsible for establishing, maintaining, and terminating connections between two or more devices. This is achieved through the use of protocols such as TCP (Transmission Control Protocol) and UDP (User Datagram Protocol).

Here is an example of how a connection is established using TCP:

```
import socket

# create a socket object
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# get local machine name
host = socket.gethostname()

# connection to hostname on the port.
s.connect((host, port))

# Receive no more than 1024 bytes
msg = s.recv(1024)

s.close()
print(msg.decode('ascii'))
```

This code creates a socket object, specifies the address family and socket type, and then connects to the specified host and port. Once the connection is established, data can be sent and received using the `send` and `recv` methods. Finally, the connection is closed using the `close` method.

Flow control in transport layer

Flow control is a mechanism used in the transport layer to prevent the sender from overwhelming the receiver with data. This is achieved by the receiver sending feedback to the sender about how much data it can receive at a given time. The sender then adjusts the rate of data transmission accordingly.

Here is an example of flow control in the transport layer using the sliding window protocol:

```

# Sender
window_size = 5
next_seq_num = 0
base = 0

while True:
    while next_seq_num < base + window_size:
        # send packet with sequence number next_seq_num
        next_seq_num += 1
    # wait for acknowledgment of packet with sequence number base
    base += 1

# Receiver
expected_seq_num = 0

while True:
    # receive packet with sequence number seq_num
    if seq_num == expected_seq_num:
        # send acknowledgment for packet with sequence number seq_num
        expected_seq_num += 1

```

etransmission in transport layer

Here is an example of code for error transmission in the transport layer:

```

def etransmission(data, error_rate):
    """
    Simulate error transmission in the transport layer.
    :param data: data to be transmitted
    :param error_rate: rate of errors during transmission
    :return: transmitted data with errors
    """
    transmitted_data = ""
    for char in data:
        if random.random() < error_rate:
            transmitted_data += chr(random.randint(0, 127))
        else:
            transmitted_data += char
    return transmitted_data

```

This function takes in the data to be transmitted and an error rate as inputs. It simulates the transmission of the data with the given error rate by randomly introducing errors into the transmitted data. The output is the transmitted data with errors.

Window management in transport layer

Window management is a key mechanism in the transport layer of the OSI model that helps regulate the flow of data between two network hosts. It is used to prevent the receiver from being overwhelmed by incoming data and to ensure that the sender does not transmit data faster than the network can handle.

Here is an example of how window management can be implemented in the transport layer using the sliding window protocol:

```
# Sender
window_size = 5
next_seq_num = 0
base = 0

def send_data(data):
    global next_seq_num
    global base
    while base < len(data):
        while next_seq_num < base + window_size and next_seq_num < len(data):
            send_packet(data[next_seq_num], next_seq_num)
            next_seq_num += 1
        receive_ack()

def receive_ack():
    global base
    ack = wait_for_ack()
    base = ack + 1

# Receiver
expected_seq_num = 0

def receive_data():
    while True:
        packet, seq_num = wait_for_packet()
        if seq_num == expected_seq_num:
            deliver_data(packet)
            send_ack(seq_num)
            expected_seq_num += 1

def send_ack(seq_num):
    # send acknowledgement for received packet
    pass
```

This is just one example of how window management can be implemented in the transport layer. There are many other approaches and variations that can be used depending on the specific requirements of the network and the application.

TCP Congestion control in transport layer

TCP congestion control is a mechanism used by the transport layer to control the flow of data in a network. It is used to prevent network congestion by regulating the amount of data that is sent into the network. Here is an example of how TCP congestion control can be implemented in code:

```
def tcp_congestion_control(window_size, threshold):
    while True:
        # Send window_size amount of data
        send_data(window_size)

        # Wait for acknowledgement
        ack = wait_for_ack()

        if ack == "success":
            # Increase window size
            window_size = min(window_size * 2, threshold)
        else:
            # Decrease window size
            window_size = max(window_size / 2, 1)
            # Decrease threshold
            threshold = max(threshold / 2, 1)
```

This code implements the Additive Increase Multiplicative Decrease (AIMD) algorithm, which is one of the algorithms used for TCP congestion control. The window size is increased when an acknowledgement is received, and decreased when a packet is lost. The threshold is also decreased when a packet is lost, to prevent the window size from growing too quickly. This helps to prevent network congestion.

Quality of service in transport layer

Quality of service (QoS) refers to the ability of a network to provide improved service to certain network traffic. The transport layer, which is responsible for end-to-end communication between applications, can provide QoS by using different transport protocols and mechanisms.

One way to provide QoS at the transport layer is through the use of the Resource Reservation Protocol (RSVP). RSVP is used by applications to request a specific level of QoS from the network for their data flows. The network then tries to reserve the necessary resources to provide the requested QoS.

Another way to provide QoS at the transport layer is through the use of the Differentiated Services (DiffServ) architecture. DiffServ uses a mechanism called “per-hop behavior” (PHB) to provide different levels of service to different traffic classes. Traffic is classified and marked at the edge of the network, and each router in the network uses the marking to determine how to handle the traffic.

In summary, the transport layer can provide QoS through the use of different protocols and mechanisms, such as RSVP and DiffServ. These mechanisms allow applications to request a specific level of service from the network, and the network tries to provide the requested service by reserving resources or by using different handling techniques for different traffic classes.

Unit 5 - Application Layer in Computer Networks

The application layer is the topmost layer in the OSI model of computer networks. This layer provides services for an application program to ensure that effective communication with another application program in a network is possible. Some of the popular application layer protocols include HTTP, FTP, SMTP, and DNS.

HTTP (Hypertext Transfer Protocol) is used for transferring web pages from a web server to a web browser. FTP (File Transfer Protocol) is used for transferring files between computers. SMTP (Simple Mail Transfer Protocol) is used for sending and receiving emails. DNS (Domain Name System) is used for resolving domain names to IP addresses.

The application layer is responsible for providing a user interface for network services. This includes functions such as sending and receiving emails, browsing the web, and transferring files. The application layer interacts with the presentation layer, which is responsible for data representation and encryption.

In summary, the application layer is responsible for providing network services to end-user applications. It provides a user interface and enables communication between applications on different computers in a network. Some of the popular application layer protocols include HTTP, FTP, SMTP, and DNS. These protocols are used for transferring web pages, files, emails, and resolving domain names to IP addresses, respectively.

Domain Name System

The Domain Name System (DNS) is a hierarchical and decentralized naming system for computers, services, or other resources connected to the Internet or a private network. It associates various information with domain names assigned to each of the participating entities. Most prominently, it translates more readily memorized domain names to the numerical IP addresses needed for locating and identifying computer services and devices with the underlying network protocols. By providing a worldwide, distributed directory service, the Domain Name System is an essential component of the functionality of the Internet.

Here is an example of a simple DNS resolution in Python:

```
import socket
addr1 = socket.gethostbyname('google.com')
addr2 = socket.gethostbyname('microsoft.com')
```

```
print(addr1)
print(addr2)
```

World Wide Web

The World Wide Web (WWW) is an information system where documents and other web resources are identified by Uniform Resource Locators (URLs, such as <https://www.example.com/>), which may be interlinked by hypertext, and are accessible over the Internet. The resources of the WWW may be accessed by users by a software application called a web browser.

Here is an example of a simple HTML code for a web page:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Web Page</title>
  </head>
  <body>
    <h1>Welcome to my web page</h1>
    <p>This is some text on my web page.</p>
  </body>
</html>
```

Hyper Text Transfer Protocol

Hyper Text Transfer Protocol (HTTP) is an application protocol for distributed, collaborative, hypermedia information systems. HTTP is the foundation of data communication for the World Wide Web, where hypertext documents include hyperlinks to other resources that the user can easily access, for example by a mouse click or by tapping the screen in a web browser.

Here is an example of a simple HTTP request and response:

```
GET / HTTP/1.1
```

```
Host: www.example.com
```

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html; charset=UTF-8
```

```
<!DOCTYPE html>
<html>
  <head>
    <title>Example</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
  </body>
```

</html>

In this example, the client sends an HTTP GET request to the server, asking for the root document (/) of the website **www.example.com**. The server responds with an HTTP response, indicating that the request was successful (200 OK) and providing the requested document in the response body.

Electronic mail in application layer

Electronic mail (email) is one of the most widely used services on the Internet. It operates at the application layer of the OSI model and uses various protocols to send and receive messages. Some of the most commonly used protocols for email are Simple Mail Transfer Protocol (SMTP), Post Office Protocol version 3 (POP3), and Internet Message Access Protocol (IMAP).

Here is an example of how email works at the application layer:

1. The user composes an email message using an email client and clicks on the send button.
2. The email client connects to the SMTP server of the sender's email service provider.
3. The SMTP server receives the message and checks the recipient's email address.
4. If the recipient's email address is on the same domain as the sender, the SMTP server delivers the message to the recipient's mailbox on the same server.
5. If the recipient's email address is on a different domain, the SMTP server forwards the message to the SMTP server of the recipient's email service provider.
6. The recipient's SMTP server receives the message and delivers it to the recipient's mailbox.
7. The recipient can then retrieve the message using an email client and either POP3 or IMAP protocol.

File Transfer Protocol in application layer

File Transfer Protocol (FTP) is a standard network protocol used to transfer files from one host to another over a TCP-based network, such as the Internet. FTP is built on a client-server architecture and uses separate control and data connections between the client and the server. Here is an example of how FTP can be implemented in Python:

```
from ftplib import FTP

ftp = FTP('ftp.example.com')
ftp.login(user='username', passwd='password')
ftp.cwd('/path/to/directory')
```

```

filename = 'example.txt'
with open(filename, 'rb') as file:
    ftp.storbinary(f'STOR {filename}', file)

ftp.quit()

```

Remote login in application layer

One way to implement remote login in the application layer is by using the Telnet protocol. Here is an example of how to use Telnet in Python to log in to a remote server:

```

import telnetlib

HOST = "your_remote_host"
user = "your_username"
password = "your_password"

tn = telnetlib.Telnet(HOST)

tn.read_until(b"login: ")
tn.write(user.encode('ascii') + b"\n")
if password:
    tn.read_until(b"Password: ")
    tn.write(password.encode('ascii') + b"\n")

tn.write(b"ls\n")
tn.write(b"exit\n")

print(tn.read_all().decode('ascii'))
tn.close()

```

This code establishes a Telnet connection to the remote host, logs in with the provided username and password, executes the `ls` command to list the contents of the current directory, and then exits the connection. The output is printed to the console.

Network management in application layer

Network management in the application layer refers to the management of network resources and services at the application level. This includes the management of applications, services, and data that are used by the network.

Here is an example of code that can be used for network management in the application layer:

```

import socket

```



```
def get_local_ip():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        s.connect(('10.255.255.255', 1))
        IP = s.getsockname()[0]
    except:
        IP = '127.0.0.1'
    finally:
        s.close()
    return IP
```

```
local_ip = get_local_ip()
print(f'Local IP address: {local_ip}')
```

This code uses the `socket` module to create a socket and connect to a remote address. The `getsockname()` method is then used to retrieve the local IP address of the machine running the code. This information can be useful for network management purposes, such as monitoring the status of network connections or configuring network settings.

Data compression in application layer

Data compression is the process of encoding information using fewer bits than the original representation. This can be achieved through various algorithms and techniques, which can be implemented at the application layer of the OSI model.

Here is an example of a simple data compression algorithm implemented in Python:

```
def compress_data(data):
    compressed_data = []
    count = 1
    prev_char = data[0]
    for char in data[1:]:
        if char == prev_char:
            count += 1
        else:
            compressed_data.append((prev_char, count))
            count = 1
            prev_char = char
    compressed_data.append((prev_char, count))
    return compressed_data
```

This algorithm takes a string of data as input and returns a list of tuples, where each tuple contains a character and the number of times it appears consecutively in the input data. This can significantly reduce the size of the data if there are many repeated characters.

Cryptography in application layer

Cryptography is used in the application layer to secure communication between applications. Here is an example of how cryptography can be implemented in the application layer using the Python programming language:

```
import hashlib
import base64

def encrypt(message: str, key: str) -> str:
    message = message.encode()
    key = hashlib.sha256(key.encode()).digest()
    encrypted_message = base64.b64encode(message)
    return encrypted_message.decode()

def decrypt(encrypted_message: str, key: str) -> str:
    encrypted_message = base64.b64decode(encrypted_message)
    key = hashlib.sha256(key.encode()).digest()
    decrypted_message = encrypted_message.decode()
    return decrypted_message
```

This code uses the `hashlib` and `base64` libraries to encrypt and decrypt messages using a key. The `encrypt` function takes a message and a key as input and returns the encrypted message. The `decrypt` function takes the encrypted message and the key as input and returns the decrypted message. The key is hashed using the SHA-256 algorithm to ensure its security.

Basic Concepts of Cryptography in Application Layer

Cryptography is the practice of securing communication in the presence of third parties. It involves the use of mathematical algorithms to convert information into a code that can only be deciphered by someone who has the key to unlock it. Cryptography is used in the application layer of the OSI model to provide security for various types of data transmission.

There are two main types of cryptography: symmetric and asymmetric. Symmetric cryptography uses the same key for both encryption and decryption, while asymmetric cryptography uses different keys for encryption and decryption.

In the application layer, cryptography is used to secure various types of communication, such as email, instant messaging, and web browsing. For example, when you send an email, it can be encrypted using a symmetric key that is shared between you and the recipient. When the recipient receives the email, they can use the same key to decrypt the message and read it.

Cryptography is also used in the application layer to secure data storage. For example, when you store data in the cloud, it can be encrypted using a key that

only you have access to. This ensures that even if someone gains access to the data, they will not be able to read it without the key.

Overall, cryptography plays a crucial role in securing communication and data storage in the application layer. It provides a means of protecting sensitive information from unauthorized access and tampering.